

## Мониторинг состояния виртуальной машины с помощью Attach API и JMX

By vmrobot on [январь 09, 2007](#)

Начиная с версии 1.5 в состав JDK входит набор [инструментов](#), позволяющих получить подробную информацию о состоянии виртуальной машины, это, например, [jstack](#) и [jconsole](#). Но что делать, если вам нужна специфическая информация, которую нельзя получить с помощью стандартных инструментов? Платформа Java предоставляет много способов для мониторинга и контроля состояния виртуальной машины с помощью интерфейсов [JVM TI](#) и [JDI](#), но для их использования виртуальная машина, состояние которой требуется контролировать, должна быть запущена с JVM TI-агентом или с определёнными опциями (такими как, например, - `agentlib:jdwp=transport=dt_shmem,server=y,suspend=n`). Но что делать, если приложение было запущено без этих опций, и необходимо получить информацию о его работе (например, узнать число работающих в приложении потоков)?

Начиная с JDK 1.6 появилась возможность запуска в рамках уже работающей виртуальной машины так называемого агента, и при этом виртуальной машине при запуске не требуется передавать никаких дополнительных опций. Эта возможность реализована с помощью Java интерфейса [Attach API](#).

Агенты, запускаемые в рамках виртуальной машины могут быть двух типов:

- агенты, написанные на Java (при этом классы агента должны быть упакованы в JAR-файл)
- традиционный JVM TI агент (то есть агент, написанный с помощью native языка, как правило представленный в виде динамически подключаемой библиотеки и использующий интерфейс JVM TI)

Хочется отметить несколько особенностей использования динамически загружаемых агентов. Если используется Java-агент, то его главный класс должен определять один из двух методов со следующими сигнатурами:

```
public static void agentmain(String agentArgs, Instrumentation inst)
```

или

```
public static void agentmain(String agentArgs)
```

Метод `agentmain` выполняется в рамках системного потока, ожидающего запросы на подключение к виртуальной машине, по этой причине `agentmain` должен содержать только код, необходимый для инициализации агента и выполняться как можно быстрее. При этом нет никаких ограничений на действия, которые могут выполняться в рамках этого метода: всё, что возможно использовать в рамках метода `main` доступно и из метода `agentmain`. Например, метод `agentmain` может просто запустить новый поток, который и будет выполнять все действия, требуемые от агента.

При использовании JVM TI агента, загружаемого в рамках уже работающей виртуальной машины, есть некоторые отличия по сравнению с JVM TI агентом, загружаемым при запуске виртуальной машины. Дело в том, что для использования практически всех функций, предоставляемых JVM TI, агент должен сначала запросить необходимую функциональность с помощью функции [AddCapabilities](#). Но часть функциональности может быть запрошена только на этапе инициализации виртуальной машины (во время работы функции агента `Agent_OnLoad`, написанной на C или C++), и поэтому не доступна динамически

загружаемому агенту.

JVMTI-агент, загружаемый с помощью Attach API, должен определять функцию `Agent_OnAttach`, в которой должна выполняться инициализация агента. Например, в момент инициализации возможно узнать о функциональности, которую может получить агент. Для этого нужно использовать функцию [GetPotentialCapabilities](#).

Вот список [возможностей](#), которые не может получить динамически загружаемый агент (эта информация была получена с использованием JDK1.6, но вполне возможно, что в следующих версиях ситуация может измениться в лучшую сторону):

- `can_generate_field_modification_events`
- `can_generate_field_access_events`
- `can_get_owned_monitor_info`
- `can_get_current_contended_monitor`
- `can_pop_frame`
- `can_get_source_debug_extension`
- `can_access_local_variables`
- `can_maintain_original_method_order`
- `can_generate_single_step_events`
- `can_generate_exception_events`
- `can_generate_frame_pop_events`
- `can_generate_breakpoint_events`
- `can_generate_method_entry_events`
- `can_generate_method_exit_events`
- `can_generate_all_class_hook_events`

(более подробно о том, что это за возможности и как их использовать, можно узнать из [спецификации JVMTI](#))

Даже несмотря на эти ограничения, в распоряжении JVMTI агента остаётся множество полезных методов для контроля состояния виртуальной машины. А какие возможности для мониторинга есть у Java агента? Один из источников информации для Java агента— возможности, предоставляемые стандартным пакетом [java.lang.management](#). Этот пакет, использующий архитектуру [Java Management Extensions \(JMX\)](#), содержит интерфейсы для мониторинга и управления виртуальной машиной.

Несколько слов о том, что такое JMX. Спецификация JMX определяет ряд пакетов, которые дают возможность использовать Java приложения в так называемых системах управления (management system) и делает платформу Java совместимой с существующими стандартами управления (например, со стандартом управления корпоративными сетями— Simple Network Management Protocol).

Главная задача системы управления—упростить работу с ресурсами, предоставляемыми приложением. Этим ресурсом может быть любой (возможно абстрактный) объект в рамках приложения, состояние которого может отслеживаться и контролироваться, например, бизнес компонент приложения или само приложение в целом. Каждый управляемый объект должен предоставлять интерфейс управления, который состоит из атрибутов объекта и операций, которые можно выполнять с объектом, посредством этого интерфейса другое приложение может манипулировать этим объектом. JMX определяет набор правил, следуя которым, можно передать Java объект под контроль системы управления, что, например, даёт возможность удалённо контролировать объект.

В рамках архитектуры JMX, ресурс, который был специальным образом инструментирован, называется MBean (Manageable Bean). Пакет [java.lang.management](#) предоставляет набор MBeans для мониторинга виртуальной машины:

- [ClassLoaderMXBean](#)—информация о системе загрузки классов
- [CompilationMXBean](#)—информация о системе компиляции
- [MemoryMXBean](#)—информация о памяти
- [ThreadMXBean](#)—информация о потоках
- [RuntimeMXBean](#)—различная информация времени выполнения
- [OperatingSystemMXBean](#)—информация об операционной системе, в которой запущена виртуальная машина
- [GarbageCollectorMXBean](#)—информация о сборке мусора
- [MemoryManagerMXBean](#)—информация о менеджере памяти
- [MemoryPoolMXBean](#)—информация о пулах памяти виртуальной машины

С помощью этих интерфейсов можно, в частности, получать уведомления о том, что количество памяти, используемой виртуальной машиной, превысило заданное значение, узнать число загруженных и выгруженных классов, или получить информацию о времени, затраченном на компиляцию и сборку мусора.

Получить экземпляр MXBean, предоставляющего информацию о виртуальной машине, в рамках той же самой виртуальной машины очень просто, достаточно одной строчки кода:

```
ThreadMXBean mxbean = ManagementFactory.getThreadMXBean();
```

и теперь с помощью ThreadMXBean можно получить довольно подробную информацию о потоках, работающих в виртуальной машине (этот интерфейс имеет около 30 методов).

Таким образом, простейший вариант использования Attach API и JMX следующий: используя Attach API загрузить Java-агент в интересующую нас виртуальную машину, агент с помощью `java.lang.management` получает необходимую информацию о виртуальной машине и каким-либо образом передаёт её приложению, запустившему агента.

Вот пример простейшего агента, который записывает информацию о потоках, работающих в виртуальной машине, в файл, имя которого передаётся агенту в качестве параметра:

```
import java.lang.instrument.*;
import java.lang.management.*;
import java.io.*;
public class AttachOnDemandAgent {
    public static void agentmain(String args, Instrumentation instr)
    throws Exception {
        PrintWriter out = new PrintWriter(new File(args));
        ThreadMXBean threadMXBean =
        ManagementFactory.getThreadMXBean();
        for (long threadID : threadMXBean.getAllThreadIds()) {
            ThreadInfo threadInfo =
            threadMXBean.getThreadInfo(threadID);
            out.println("Thread: " + threadInfo.getThreadName() + " "
            + threadInfo.getThreadState());
        }
        out.close();
    }
}
```

Классы агента упаковываем в jar-архив, при этом манифест архива должен содержать атрибут `Agent-Class` с именем главного класса агента.

Для того, чтобы запустить агент в рамках интересующей нас виртуальной машины, используем [Attach API](#). Сначала надо получить экземпляр [com.sun.tools.attach.VirtualMachine](#), для этого необходимо узнать идентификатор процесса виртуальной машины (кстати, можно использовать утилиту [jps](#), входящую в состав JDK).

Простейшая программа, которая загружает агент, находящийся в файле `agent.jar`, в виртуальную машину, идентификатор процесса которой передаётся в качестве параметра (для компиляции и запуска программы необходим файл `<JDK>/lib/tools.jar`, кстати, при использовании классов из пакетов `com.sun.*`, надо помнить, что эти классы могут меняться на усмотрение Sun, так что использовать их надо с осторожностью):

```
import java.io.File;
import com.sun.tools.attach.*;
public class AttachAgent {
    public static void main(String[] args) {
        try {
            // создаём экземпляр VirtualMachine на основе
            // идентификатора процесса, переданного в качестве параметра
            VirtualMachine vm = VirtualMachine.attach(args[0]);
            // виртуальная машина, в которую загружается агент, должна
            // быть способна найти JAR файл с агентом
            // и файл, в который агент выводит результат работы,
            // поэтому функции VirtualMachine.loadAgent
            // передаются полные имена файлов
            File agentJarFile = new File("agent.jar");
            File outputFile = new File("agent.out");

            vm.loadAgent(agentJarFile.getAbsolutePath(),
            outputFile.getAbsolutePath());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Командная строка для присоединения агента к работающей виртуальной машине может выглядеть так (файл `agent.jar` с запакованным классом `AttachOnDemandAgent` должен находиться в рабочей папке программы):

```
java AttachAgent 5609
```

(здесь 5609 - идентификатор процесса виртуальной машины, в которой должен запускаться агент).

Приведённый пример демонстрирует, насколько просто можно получить доступ к информации о состоянии виртуальной машины. Конечно, возможности JMX не ограничиваются тем, что информацию, предоставляемую MBean'ами можно просто вывести в файл. Например, агент, запущенный в рамках интересующей виртуальной машины, может запустить MBean-сервер, к которому клиент может подключиться удалённо и получить информацию об интересующих его ресурсах. Текущая реализация JMX предоставляет реализацию сервера и клиента на основе RMI. Я же хочу показать пример удалённого доступа к MBean, предоставляемым пакетом `java.lang.management` по протоколу HTTP.

Реализация JMX 1.2, доступная [здесь](#), содержит класс `HtmlAdaptorServer`, который предоставляет доступ к MBean серверу через веб-браузер (при компиляции и запуске необходим файл `JXM/lib/jxmtools.jar`).

Изменим код агента следующим образом:

```
import java.lang.instrument.*;
import java.lang.management.*;
import javax.management.*;
import com.sun.jdmk.comm.HtmlAdaptorServer;
```

```

public class AttachOnDemandAgent {
    public static void agentmain(String args, Instrumentation instr)
    throws Exception {
        // получаем MBean сервер
        MBeanServer server =
        ManagementFactory.getPlatformMBeanServer();

        // создаём HtmlAdaptorServer доступный на указанном порту
        int port = 8090;
        HtmlAdaptorServer htmlAdaptorServer = new
        HtmlAdaptorServer(port);
        // регистрируем и запускаем HtmlAdaptorServer
        ObjectName name = null;
        name = new ObjectName("Adaptor:name=HtmlAdaptorServer,port=" +
        port);
        server.registerMBean(htmlAdaptorServer, name);

        htmlAdaptorServer.start();
    }
}

```

Надо заметить, что при запуске этого агента, виртуальная машина в рамках которой работает агент, должна быть способна найти все классы, используемые агентом, поэтому необходимо либо поместить необходимые файлы из `jxmtools.jar` в архив агента, либо путь к классам виртуальной машины на этапе загрузки, в которой будет работать агент, должен содержать `jxmtools.jar` (см. опцию `-Xbootclasspath`).

После запуска агента указываем браузеру адрес (например `http://localhost:8090`) и получаем интерактивный доступ к виртуальной машине из веб браузера! Например, вот так выглядит `ClassLoaderMXBean`:

#### List of MBean attributes:

| Name                                  | Type    | Access | Value                                                             |
|---------------------------------------|---------|--------|-------------------------------------------------------------------|
| <a href="#">LoadedClassCount</a>      | int     | RO     | 2482                                                              |
| <a href="#">TotalLoadedClassCount</a> | long    | RO     | 2674                                                              |
| <a href="#">UnloadedClassCount</a>    | long    | RO     | 192                                                               |
| <a href="#">Verbose</a>               | boolean | RW     | <input type="radio"/> True <input checked="" type="radio"/> False |

Apply